GUIA VISUAL 073

ARQUIVOS, DESCRITORES E PROCESSOS EM SCRIPTS

*O analista queria só deixar a lista de alvos ordenada no lugar: `sort alvos.txt > alvos.txt`. O arquivo ficou com **zero bytes** — sem mensagem de erro, sem confirmação, e o backup mais recente era de três dias antes. A explicação não está no `sort`: está na **ordem em que o shell faz as coisas**. Ele prepara os redirecionamentos **antes** de executar o comando, e `> arquivo` quer dizer "abra truncando". Quando o `sort` foi ler, o arquivo já estava vazio havia milissegundos.*

073
GUIAS LINUX

A TABELA DE DESCRITORES · E A ORDEM QUE ESVAZIOU O ARQUIVO

SEU SCRIPT
bash backup.sh
PID 4821

0	stdin	→	/dev/pts/1 o terminal
1	stdout	→	saida.txt exec 1> saida.txt
2	stderr	→	/dev/pts/1 o terminal
3	livre	→	dados.csv exec 3< dados.csv
9	livre	→	rastro.log BASH_XTRACEFD=9
200	livre	→	meu.lock flock -n 200

AS FORMAS DE REDIRECIONAR

`> arq`	abre para escrita, **truncando**
`>> arq`	escreve acrescentando ao fim
`< arq`	abre para leitura
`<> arq`	leitura **e** escrita, sem truncar
`>| arq`	escreve mesmo com `noclobber` ligado
`2>&1`	o 2 passa a apontar para onde o 1 aponta **naquele momento**
`&> arq`	atalho do Bash para `> arq 2>&1`
`n> arq`	redireciona o descritor n; `n>&m` duplica; `n>&-` fecha

`sort a.txt > a.txt`

1	2	3	4
shell prepara o redirecionamento	a.txt **truncado**	sort só agora começa a rodar	lê 0 bytes, grava 0 bytes — **sem aviso**

`sort a > tmp && mv tmp a`

1	2	3	4
mktemp **no mesmo diretório**	sort lê o original, grava no temporário	o **&&** só segue se deu certo	mv renomeia **atomicamente**

A regra vale para qualquer transformação de arquivo, não só o `sort`: leia de um lugar, escreva em outro, troque no fim. O ponto de atomicidade é o `mv` **dentro do mesmo sistema de arquivos** — entre sistemas diferentes ele vira cópia mais remoção, e o intervalo entre as duas é exatamente a janela que se queria eliminar. Por isso o temporário nasce ao lado do original, e não em `/tmp`. Se o `sort` falhar, o `&&` impede o `mv` e o original fica intacto. Alternativas: `sponge` (do `moreutils`) absorve toda a entrada antes de abrir o destino — `sort alvos.txt | sponge alvos.txt` — ao custo de manter tudo em memória; e `sed -i` / `gawk -i inplace` resolvem, mas **não são atômicos de verdade**: gravam num temporário e renomeiam, o que troca o inode e **quebra hard links**. Como rede extra, `set -o noclobber` transforma sobrescrita acidental em erro, e `>|` é a permissão explícita. E a pegadinha da ordem: `cmd > log 2>&1` manda os dois para o arquivo, mas `cmd 2>&1 > log` manda o **stderr para o terminal** — quando o `2>&1` é processado, o 1 ainda aponta para lá. Leia da esquerda para a direita, como uma sequência de atribuições.

LER LINHA A LINHA · EXEC · FLOCK

kali@kali: ~/lab-descritores

```
# o laço canônico — cada peça corrige um defeito
$ while IFS= read -r linha || [[ -n "$linha" ]]; do
    printf '[%s]\n' "$linha"
  done < entrada.txt
# comando de dentro que lê stdin engole o resto: use outro fd
$ while IFS= read -r -u 3 host; do ssh "$host" uptime; done 3< hosts.txt
# carregar tudo num array de uma vez
$ mapfile -t alvos < <(grep -v '^#' escopo.txt)
$ printf 'total: %d\n' "${#alvos[@]}"
total: 42
# registrar TUDO sem perder a tela — uma linha no topo do script
$ exec > >(tee -a /var/log/meuscript.log) 2>&1
# salvar e restaurar o stdout
$ exec 3>&1; exec 1> /tmp/relatorio.txt; echo linha; exec 1>&3 3>&-
# trava: nada de arquivo .pid — o kernel solta até em kill -9
$ exec 200> /var/lock/meu.lock
$ flock -n 200 || { echo "ja ha uma execucao" >&2; exit 1; }
# dois consumidores da mesma saída, e um teste de porta sem nc
$ gerar | tee >(gzip > r.gz) >(sha256sum > r.sha) > r.txt
$ timeout 2 bash -c 'exec 3<>/dev/tcp/127.0.0.1/22' && echo "22 aberta"
22 aberta
```

Higiene do temporário, em quatro linhas: `umask 077` **antes** de criar; respeite o `TMPDIR` (em serviço systemd com `PrivateTmp=yes` ele aponta para um diretório isolado); `TRABALHO="$(mktemp -d -t meuscript.XXXXXXXX)"`, com molde nomeado para o diretório ser identificável no `ls /tmp` quando algo der errado; e `trap 'rm -rf -- "$TRABALHO"' EXIT`. Para segredo que não deve tocar o disco, `mktemp -p /dev/shm` — memória, sem rastro em disco, mas **ainda legível por quem tem privilégio** na máquina.

O LAÇO CANÔNICO, PEÇA POR PEÇA

`IFS=`	vazio, impede que **espaços e tabulações das bordas** sejam removidos
`-r`	não interpreta a barra invertida — sem ele, `C:\novo` vira `C:ovo`
`|| [[ -n ]]`	salva a **última linha sem quebra final**: sem isso ela é lida mas o `read` devolve 1 e o laço a descarta em silêncio
`< arq no done`	e nunca `cat arq |`: o pipe cria subshell e **toda variável modificada no laço se perde**. Para saída de comando, `done < <(cmd)`

EXEC E FLOCK

`exec`	sem comando, aplica o redirecionamento ao **shell inteiro** — todos os comandos seguintes herdam
`flock -n`	falha na hora em vez de esperar; sem `-n`, a segunda cópia fica na fila. **Não precisa apagar o lock:** o kernel libera até em `SIGKILL` — é a vantagem sobre o arquivo `.pid`, que deixa órfão em toda morte anormal
`<( ... )`	vira **caminho de arquivo** (`/dev/fd/63`), o que alimenta programa que não aceita pipe: `diff <(sort a) <(sort b)`
`>( ... )`	o shell **não espera** o processo de dentro terminar — ler o arquivo na linha seguinte pode pegá-lo incompleto. Use `wait`
`/dev/tcp`	socket implementado **pelo próprio Bash**: não existe no sistema de arquivos e não funciona em **dash** nem **sh**

O SCRIPT COMO PROCESSO

`$$`	PID do shell atual — **não muda** em subshell
`BASHPID`	PID real, que **muda** em subshell
`$! · PPID`	último posto em segundo plano · o pai
`BASH_SOURCE`	`[[ "${BASH_SOURCE[0]}" == "$0" ]]` distingue **executado** de **carregado com `source`** — é o que deixa o mesmo arquivo ser biblioteca e programa